Algebra and Discrete Mathematics (ADM)

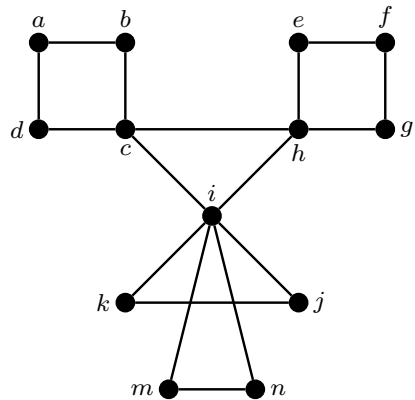
Tutorial 9 Hamiltonian cycles

Lecturer: doc. Xiaolu Hou, PhD.
xiaolu.hou@stuba.sk

Necessary conditions for existence of Hamiltonian cycles

- G must be connected
- No vertex of G can have degree less than 2
- G cannot contain a cut-vertex

The last condition is not satisfied: i, h, and c are cut-vertices.



Existence of Hamiltonian cycles

Necessary conditions:

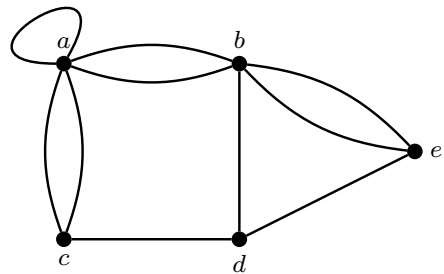
- G must be connected
- No vertex of G can have degree less than 2
- G cannot contain a cut-vertex

A sufficient condition (Dirac's Theorem):

- The graph has $n \geq 3$ vertices.
- Every vertex v of G satisfies $\deg(v) \geq \dfrac{n}{2}$

$$n = 5, \quad \frac{n}{2} = 2.5$$

One Hamiltonian cycle is $abedca$.



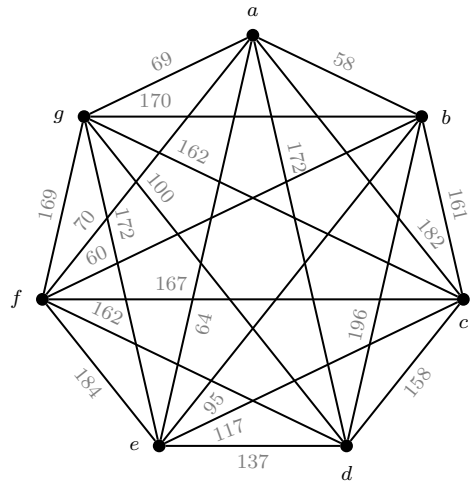
Nearest Neighbor Algorithm

1. Choose a starting vertex, say v. Highlight v.
2. Among all edges incident to v, choose one with the smallest weight. If more than one edge has the same weight, choose one arbitrarily.
3. Highlight the chosen edge and move to its other endpoint u. Highlight u.
4. Repeat Steps 2 and 3, considering only edges leading to unhighlighted vertices.
5. Close the cycle by adding the edge from the last highlighted vertex back to v. Then calculate the total weight.

Nearest Neighbor Algorithm

- Step 1. We start from vertex a
- Step 2. Highlight edge ab
- Step 2. Highlight edge bf
- Step 2. We cannot choose fa or fb, so we highlight edge fd.
- Step 2. Highlight edge dg
- Step 2. Highlight edge gc
- Step 2. Highlight edge ce
- step 5. Close the cycle by adding ea

$$a \to b \to f \to d \to g \to c \to e \to a$$

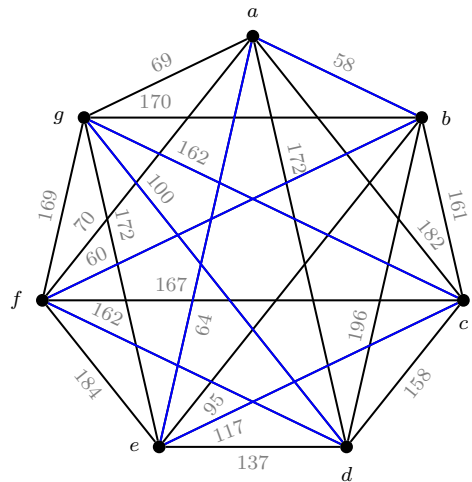


Nearest Neighbor Algorithm

- Step 1. We start from vertex a
- Step 2. Highlight edge ab
- Step 2. Highlight edge bf
- Step 2. Cannot choose fa, fb, highlight edge fd
- Step 2. Highlight edge dg
- Step 2. Highlight edge gc
- Step 2. Highlight edge ce
- Step 5. Close the cycle by adding ea

$$a \to b \to f \to d \to g \to c \to e \to a$$

- Total weight: 723



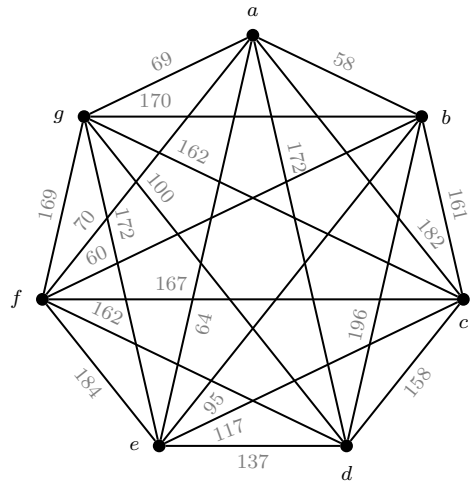
Cheapest Link Algorithm

1. Among all edges in the graph, choose one with the smallest weight. If more than one edge has the same weight, choose one arbitrarily. Highlight the chosen edge.
2. Repeat Step 1 subject to the following conditions:
   - no vertex has three highlighted edges incident to it
   - no edge is chosen if it would create a cycle before all vertices are included
3. Calculate the total weight

Cheapest Link Algorithm

- Step 1. Choose edge ab
- Step 1. Choose edge bf
- Step 1. Choose edge ae
- Step 1. Cannot choose edge ag, otherwise a would have three edges incident to it. Choose edge dg
- Step 1. Choose edge ce
- Step 1. Choose edge cd
- Close the cycle by adding edge fg.

$$a \to b \to f \to g \to d \to c \to e \to a$$

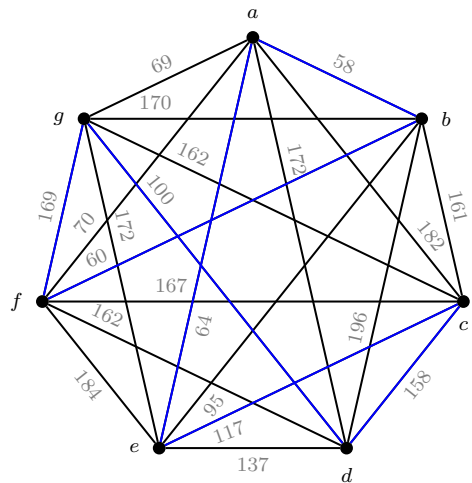


Cheapest Link Algorithm

- Step 1. Choose edge ab
- Step 1. Choose edge bf
- Step 1. Choose edge ae
- Step 1. Cannot choose edge ag, otherwise a would have three edges incident to it. Choose edge dg
- Step 1. Choose edge ce
- Step 1. Choose edge cd
- Close the cycle fg

$$a \to b \to f \to g \to d \to c \to e \to a$$

- Total weight: 726



Nearest Insertion Algorithm

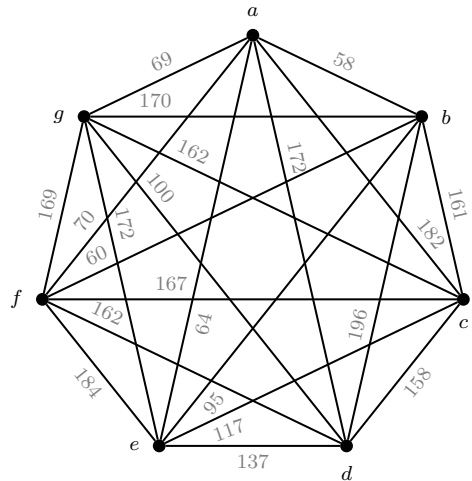
1. Choose an edge of smallest weight. Highlight the edge and its endpoints.
2. Choose a vertex closest to one of the two highlighted vertices. Highlight this new vertex and the two edges joining it to the highlighted vertices.
3. Identify the unvisited vertex closest to the current cycle. Insert it into the cycle by adding two edges and removing one existing edge. Choose the insertion with the smallest increase in total weight.
4. Repeat Step 3 until all vertices have been included in the cycle.
5. Calculate the total weight.

Nearest Insertion Algorithm

- Step 1. Choose edge ab
- Step 2. Pick vertex f, add edges fb, fa.
- Step 3. Pick vertex e, add edge be. We need to decide whether to add edge ae and delete ab, or add edge fe and delete bf.

$$\begin{aligned} \omega(ae) - \omega(ab) &= 64 - 58 = 6 \\ \omega(fe) - \omega(bf) &= 184 - 60 = 124 \end{aligned}$$

Hence, we add ae and delete ab.

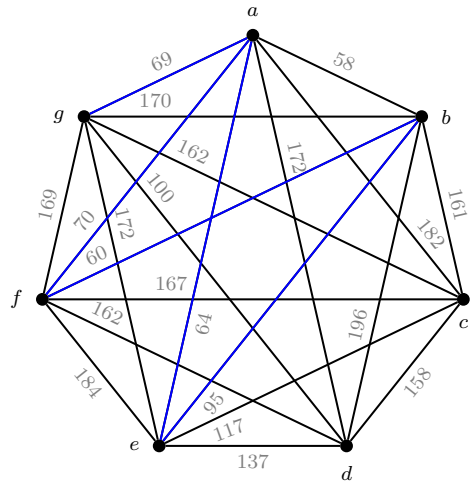


Nearest Insertion Algorithm

- Step 3. Pick vertex g, add edge ga. We need to decide whether to add edge gf and delete af, or add edge ge and delete ae.

$$\begin{aligned} \omega(gf) - \omega(af) &= 169 - 70 = 99 \\ \omega(ge) - \omega(ae) &= 172 - 64 = 108 \end{aligned}$$

Hence, we add gf and delete af.

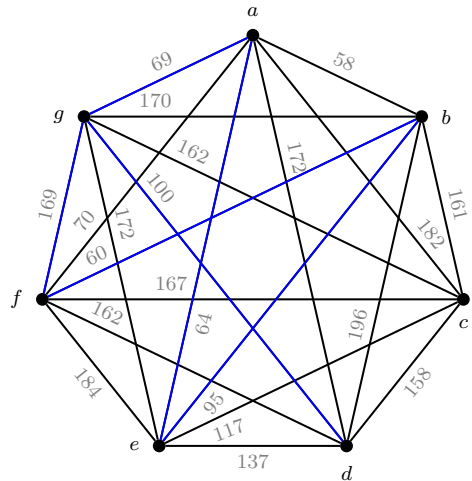


Nearest Insertion Algorithm

- Step 3. Pick vertex d, add edge gd. We need to decide whether to add edge da and delete ag, or add edge df and delete gf.

$$\begin{aligned} \omega(da) - \omega(ga) &= 172 - 69 = 103 \\ \omega(df) - \omega(gf) &= 162 - 169 = -7 \end{aligned}$$

Hence, we add df and delete gf.

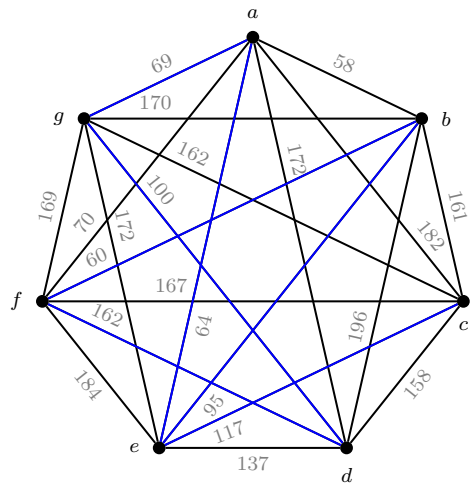


Nearest Insertion Algorithm

- Step 3. Pick vertex c, add edge ce. We need to decide whether to add edge ca and delete ea, or add edge cb and delete eb.

$$\begin{aligned} \omega(ca) - \omega(ea) &= 182 - 64 = 118 \\ \omega(cb) - \omega(eb) &= 161 - 95 = 66 \end{aligned}$$

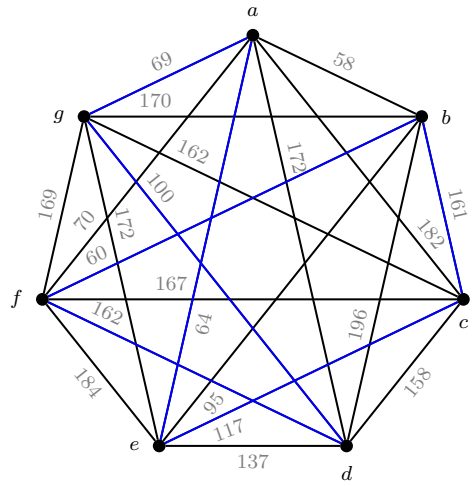
Hence, we add cb and delete eb.



Nearest Insertion Algorithm

$$a \to g \to d \to f \to b \to c \to e \to a$$

Total weight: 733



Undirecting Algorithm

Input: Weighted complete digraph $G = (V, A, \omega)$

Steps:

1. For each vertex x, create a clone x'. Form the edge xx' with weight 0.
2. For each arc xy, form the edge $x'y$.
3. Assign each edge the weight of its corresponding arc.
   - $\omega(xx') = 0$
   - $\omega(x'y) = \omega(xy)$
   - $\omega(xy') = \omega(yx)$

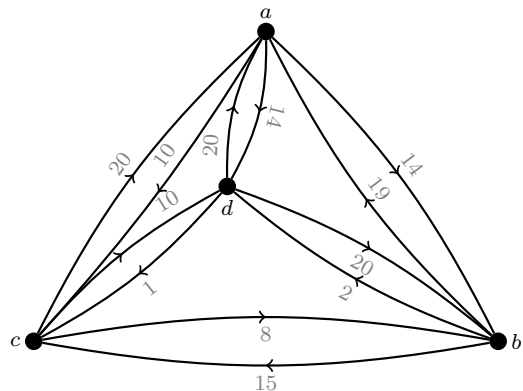
Output: weighted clone graph $G' = (V', E, \omega')$

Note

- We need to ensure that when forming a cycle an arc into a vertex is immediately followed by an arc out of that same vertex
- By giving the edge between a vertex and its clone a weight of 0 (when all other arcs have positive weights), we ensure this edge will always be included in any Hamiltonian cycle.
- In addition, this extra edge does not impact the total weight of the final cycle

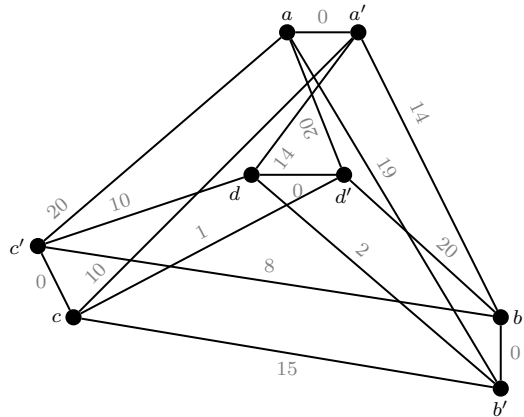
Undirecting Algorithm

	a	b	c	d
a	.	14	10	14
b	19	.	15	2
c	20	8	.	10
d	20	20	1	.

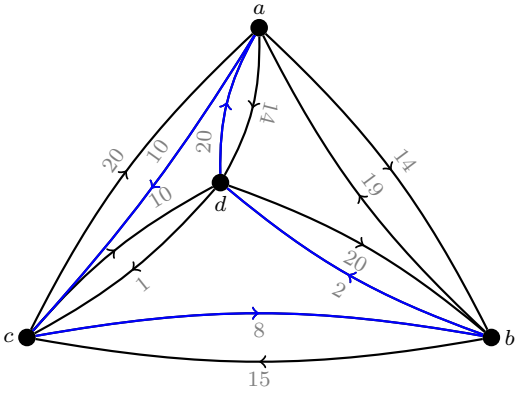


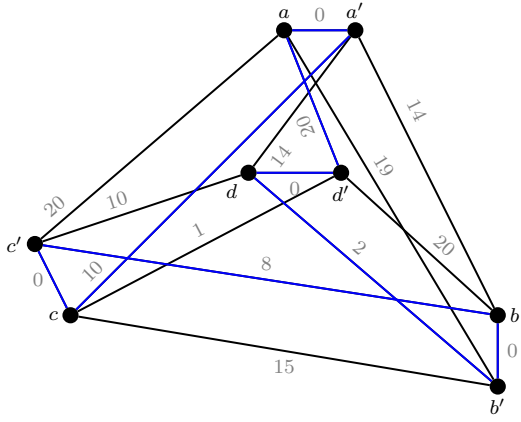
Undirecting Algorithm

	a	b	c	d	a'	b'	c'	d'
a	.	.	.	.	0	19	20	20
b	.	.	.	.	14	0	8	20
c	.	.	.	.	10	15	0	1
d	.	.	.	.	14	2	10	0
a'	0	14	10	14	.	.	.	.
b'	19	0	15	2	.	.	.	.
c'	20	8	0	10	.	.	.	.
d'	20	20	1	0	.	.	.	.



Nearest Neighbor Algorithm





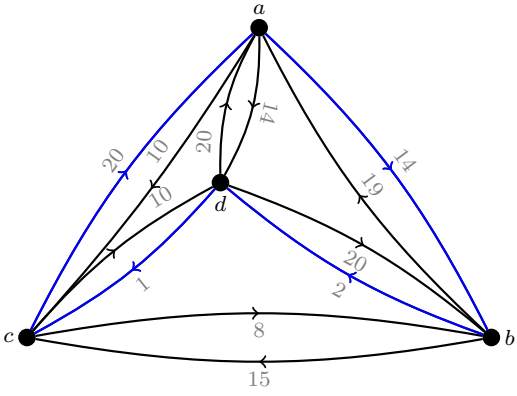
- Nearest Neighbor Algorithm

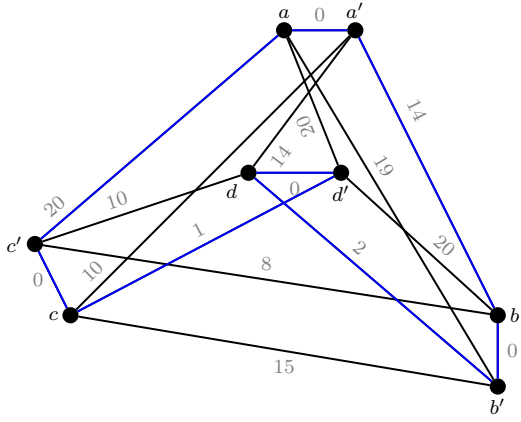
$$aa'cc'bb'dd'a$$

- Convert this cycle back to the digraph:

$$a \to c \to b \to d \to a$$

Cheapest Link Algorithm





- Cheapest Link Algorithm

$$aa'bb'dd'cc'a$$

- Convert to the digraph

$$a \to b \to d \to c \to a$$